FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering

Academic Year 2025-2026

Rubrics for Lab Exp eriments

Class: B.E. Computer Engineering **Subject Name:** ML

Semster: VIII **Subject Code:** CSC701

Practical No.	8
Title:	To Study and implement Dimension Reduction
Date of Performance:	17-10-2025
Roll No.	9913
Student	Mark Lopes

Evaluation

Performance Indicator	Very Poor	Poor	Average	Good	Excellent
Timeline (2)	More than three sessions late(0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or on time(2)
Efforts (4)	N/A	N/A	Not completed(1)	Partially completed(1)	Completed(4)
Legibility (2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially understood the concept(1)	Understood the concept(2)
Total					

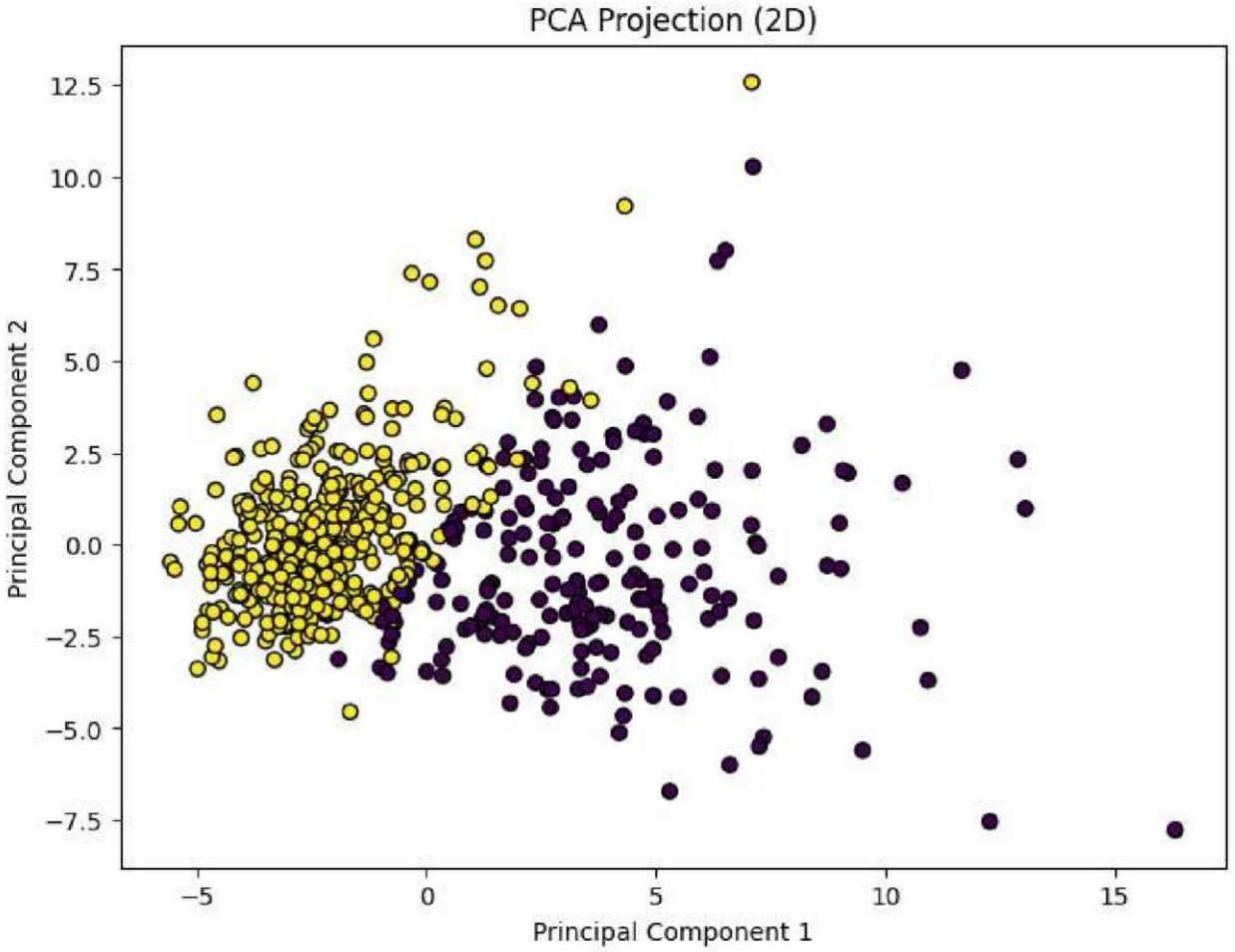
Signature of the Teacher:

```
# Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris, load_breast_cancer
from sklearn.decomposition import PCA
from sklearn.feature_selection import VarianceThreshold
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

# ---------------------------------------------
# Load Dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print("Original Dataset Shape:", X.shape)

# ---------------------------------------------
# 1. Principal Component Analysis (PCA)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', edgecolor='k')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Projection (2D)')
plt.show()

# ---------------------------------------------
# 2. Low Variance Filter
selector = VarianceThreshold(threshold=0.01)
X_low_var = selector.fit_transform(X)
print("Shape after Low Variance Filter:", X_low_var.shape)

# ---------------------------------------------
# 3. High Correlation Filter
def correlation_filter(data, threshold=0.9):
    corr_matrix = data.corr().abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bo
    to_drop = [column for column in upper.columns if any(upper[column] > thresho
```

```
        return data.drop(to_drop, axis=1)

    X_uncorr = correlation_filter(X)
    print("Shape after High Correlation Filter:", X_uncorr.shape)

    # -----------------------------------------
    # 4. Feature Selection using Random Forest
    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X, y)
    feature_importances = pd.Series(model.feature_importances_, index=X.columns)
    top_features = feature_importances.nlargest(10).index
    X_rf = X[top_features]

    print("Top 10 Features from Random Forest:\n", top_features)

    # -----------------------------------------
    # 5. Autoencoder for Dimensionality Reduction
    # Normalize input
    X_norm = StandardScaler().fit_transform(X)

    # Define Autoencoder
    input_dim = X.shape[1]
    encoding_dim = 10  # reduced dimensions

    input_layer = Input(shape=(input_dim,))
    encoded = Dense(encoding_dim, activation='relu')(input_layer)
    decoded = Dense(input_dim, activation='sigmoid')(encoded)

    autoencoder = Model(input_layer, decoded)
    encoder = Model(input_layer, encoded)

    autoencoder.compile(optimizer='adam', loss='mse')

    # Train autoencoder
    autoencoder.fit(X_norm, X_norm, epochs=50, batch_size=32, shuffle=True, verbose=
```

```
    # Encode data
    X_encoded = encoder.predict(X_norm)
    print("Shape after Autoencoder Encoding:", X_encoded.shape)

    # Visualize encoded 2D space if encoding_dim = 2
    if encoding_dim == 2:
        plt.scatter(X_encoded[:, 0], X_encoded[:, 1], c=y, cmap='coolwarm', edgecolo
        plt.title("2D Encoded Space via Autoencoder")
        plt.xlabel("Encoded Feature 1")
        plt.ylabel("Encoded Feature 2")
        plt.show()
```

```
Original Dataset Shape: (569, 30)
```




```
Shape after Low Variance Filter: (569, 14)
Shape after High Correlation Filter: (569, 20)
Top 10 Features from Random Forest:
 Index(['worst area', 'worst concave points', 'mean concave points',
       'worst radius', 'worst perimeter', 'mean perimeter', 'mean concavity',
       'mean area', 'worst concavity', 'mean radius'],
      dtype='object')
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step
Shape after Autoencoder Encoding: (569, 10)
```

Univariate EDA

```
# Step 1: Import required libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA

# Step 2: Load dataset
from sklearn.datasets import load_iris
```

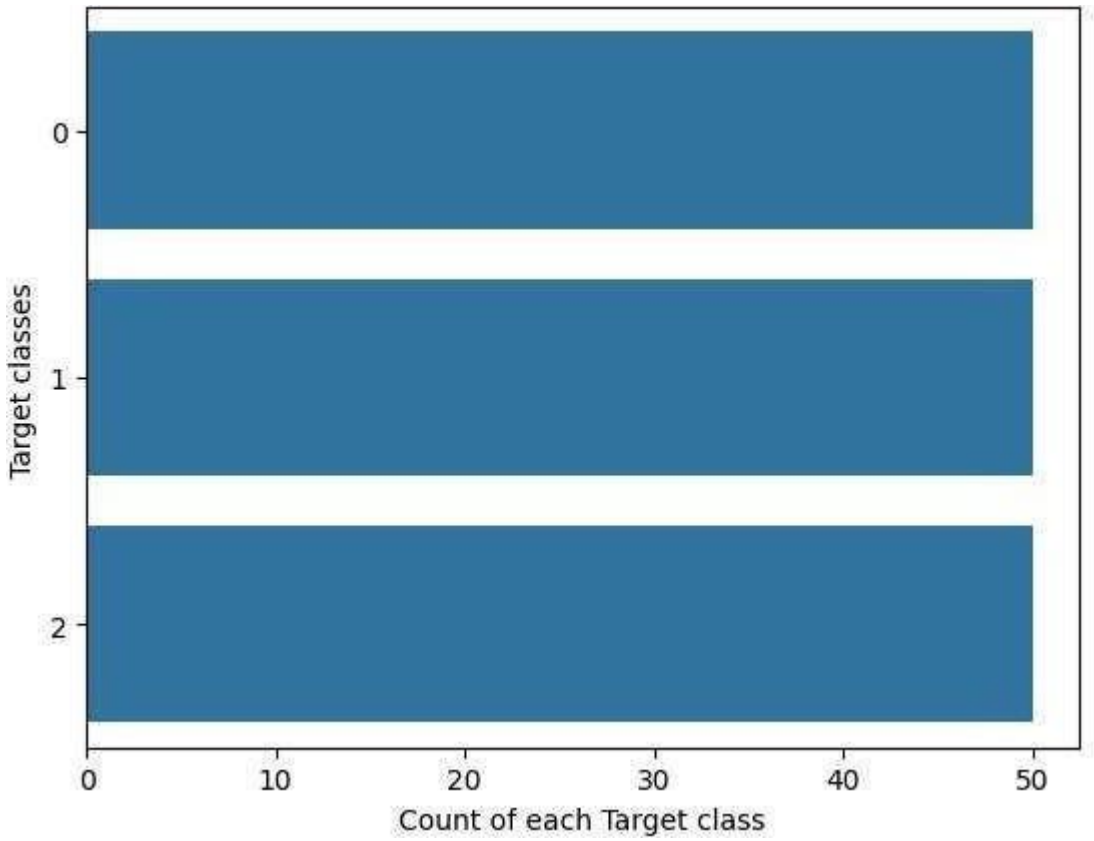
```
iris_data = load_iris()
X = pd.DataFrame(data=iris_data.data, columns=iris_data.feature_names)
y = pd.Series(iris_data.target, name='target')


# Step 3: Basic Info
print("Dataset Info:")
print(X.info())

# Step 4: Summary Statistics
print("\nDescriptive Statistics:")
print(X.describe())

# Step 5: First 5 rows
print("\nSample Data:")
print(X.head())
```

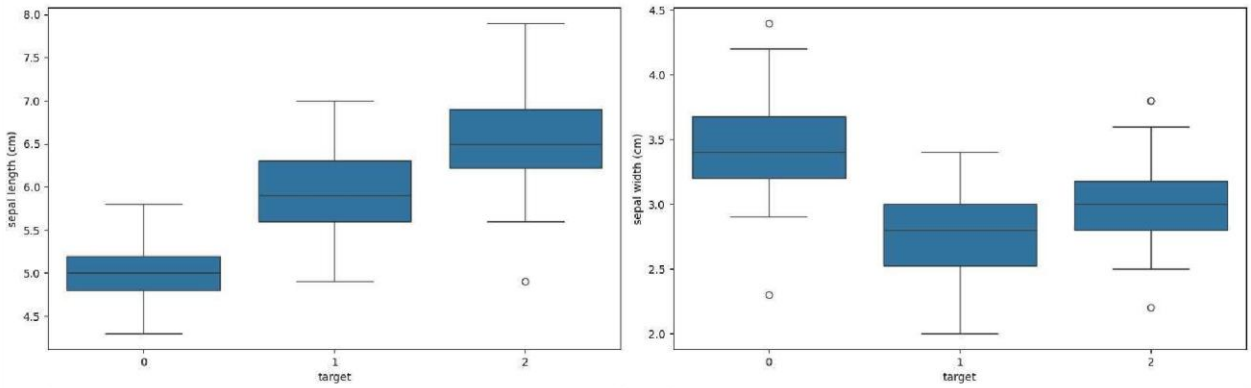
```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
dtypes: float64(4)
memory usage: 4.8 KB
None

Descriptive Statistics:
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000   
mean            5.843333          3.057333           3.758000   
std             0.828066          0.435866           1.765298   
min             4.300000          2.000000           1.000000   
25%             5.100000          2.800000           1.600000   
50%             5.800000          3.000000           4.350000   
75%             6.400000          3.300000           5.100000   
max             7.900000          4.400000           6.900000   

       petal width (cm)  
count        150.000000  
mean           1.199333  
std            0.762238  
min            0.100000  
25%            0.300000  
50%            1.300000  
75%            1.800000  
max            2.500000  

Sample Data:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
```

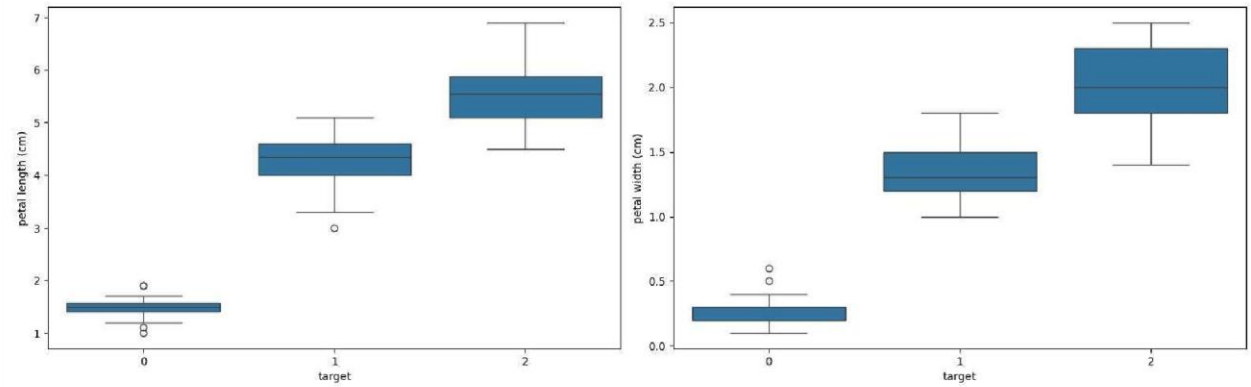
```
4                5.0               3.6               1.4               0.2
```

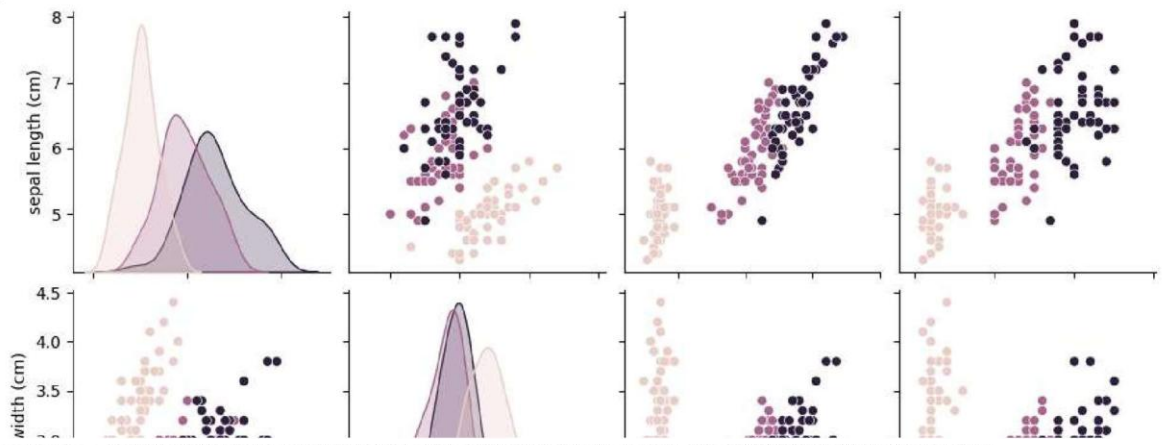
Bivariate EDA and Encoding

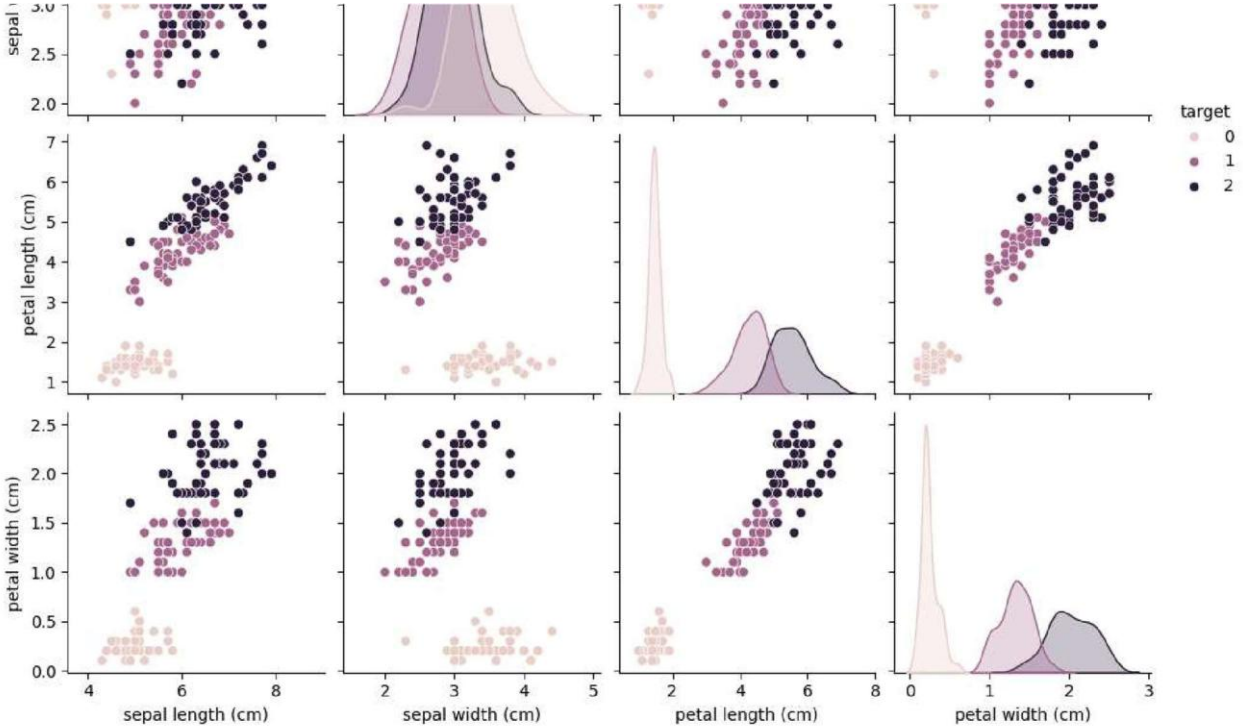
```
# Encode target labels (This step is not needed when using the target from sklea
# iris_data['species'] = iris_data['species'].replace({'Iris-setosa': 0, 'Iris-v

# Count plot
sns.countplot(y=y) # Use the 'y' variable for the target
plt.xlabel("Count of each Target class")
plt.ylabel("Target classes")
plt.show()

# Box plots by species
fig, ax = plt.subplots(nrows=2, ncols=2, figsize=(16, 10))
features = X.columns # Use the column names from the features DataFrame 'X'

for i, feature in enumerate(features):
    row, col = i // 2, i % 2
    sns.boxplot(x=y, y=X[feature], ax=ax[row][col]) # Use 'y' for the x-axis and
plt.tight_layout()
plt.show()

# Pairplot
# sns.pairplot(iris_data, hue='species') # This will not work directly with the
# Let's create a DataFrame with both features and target for the pairplot
iris_df = pd.concat([X, y], axis=1)
sns.pairplot(iris_df, hue='target')
plt.show()


# Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(iris_df.corr(), annot=True, cmap='RdYlGn') # Use the combined datafr
plt.title("Feature Correlation Matrix")
plt.show()

# Histograms
X.hist(figsize=(12, 10), bins=15) # Use the features DataFrame 'X'
plt.suptitle("Feature Distributions")
plt.show()
```

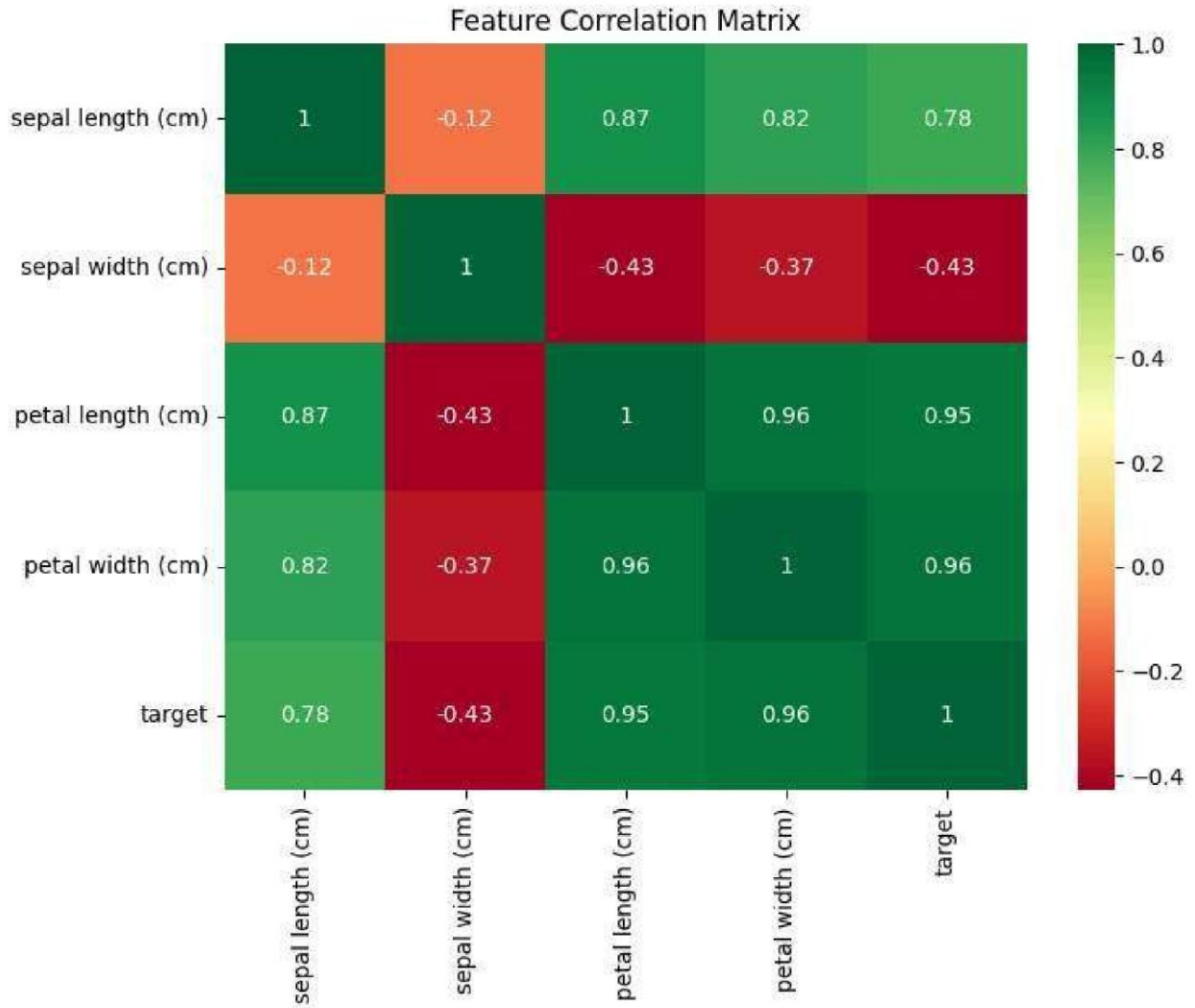

Target classes
0
1
2
0
10
20
30
40
50
Count of each Target class


sepal length (cm)
target
sepal width (cm)
target

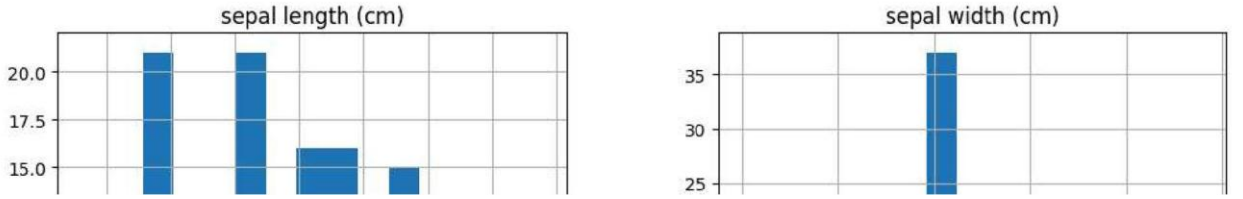

petal length (cm)
target
petal width (cm)
target

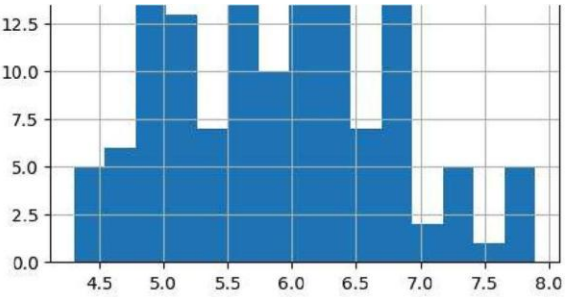

sepal length (cm)
width (cm)

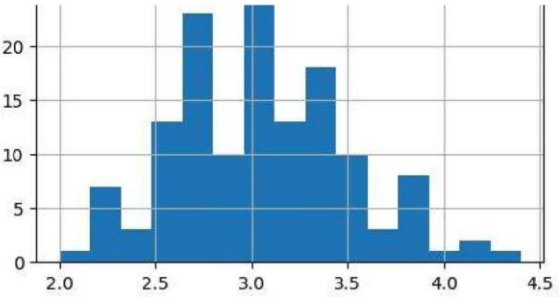


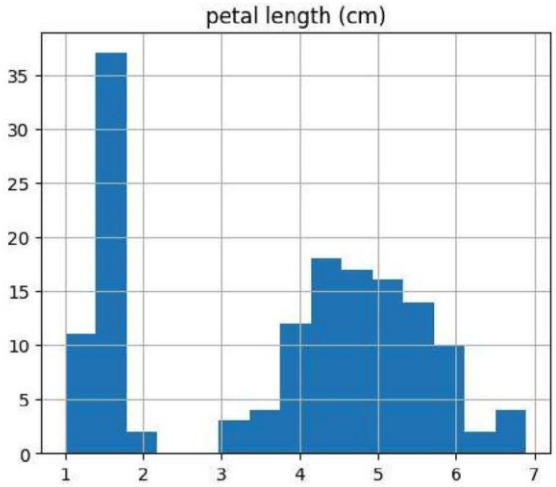


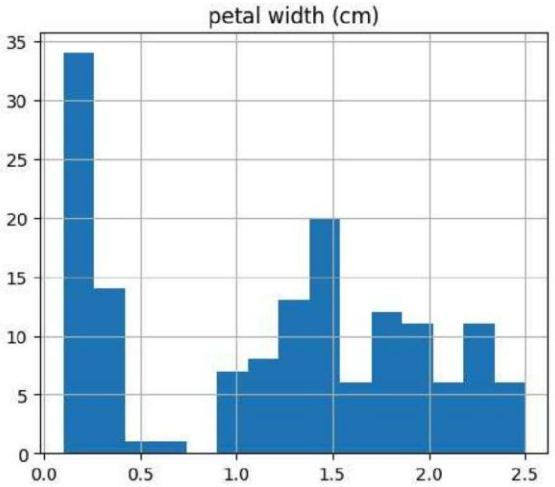
Feature Distributions




12.5
10.0
7.5
5.0
2.5
0.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0


20
15
10
5
0
2.0
2.5
3.0
3.5
4.0
4.5


petal length (cm)
35
30
25
20
15
10
5
0
1
2
3
4
5
6
7


petal width (cm)
35
30
25
20
15
10
5
0
0.0
0.5
1.0
1.5
2.0
2.5

🔹 Modelling 1️⃣ Without PCA

```
# Separate features and target - using X and y already created
# X = iris_data.drop('species', axis=1)
# y = iris_data['species']

# Normalize
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,

# Train KNN without PCA
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)

# Accuracy scores
print("Train score before PCA:", knn.score(X_train, y_train))
print("Test score before PCA:", knn.score(X_test, y_test))
```

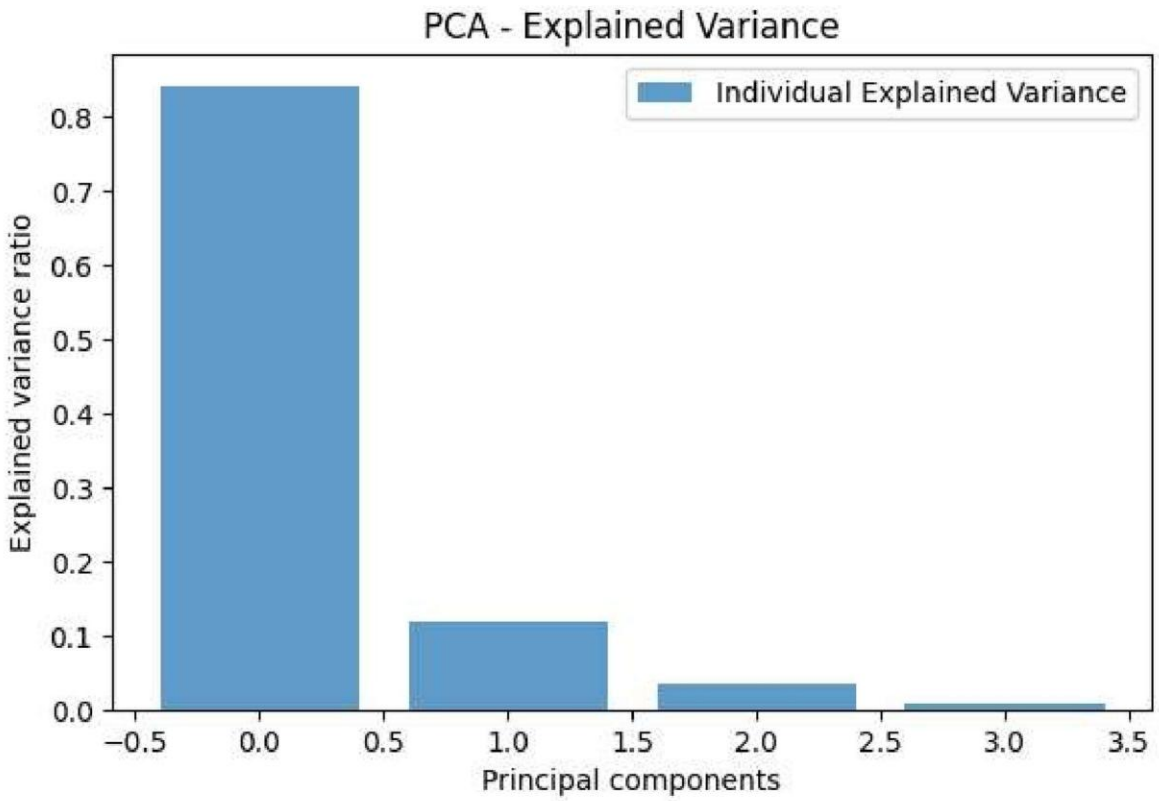
```
Train score before PCA: 0.9428571428571428
Test score before PCA: 1.0
```

2️⃣ With PCA

```
# Apply PCA
pca = PCA()
X_pca_full = pca.fit_transform(X_scaled)

# Explained Variance
explained_variance = pca.explained_variance_ratio_
print("Explained Variance Ratio:", explained_variance)

# Plot variance
plt.figure(figsize=(6, 4))
plt.bar(range(4), explained_variance, alpha=0.7, align='center', label='Individu
plt.ylabel('Explained variance ratio')
plt.xlabel('Principal components')
plt.legend(loc='best')
plt.tight_layout()
plt.title("PCA - Explained Variance")
plt.show()

# PCA with 3 components
pca = PCA(n_components=3)
X_reduced = pca.fit_transform(X_scaled)

# Train-Test split
X_train_pca, X_test_pca, y_train, y_test = train_test_split(X_reduced, y, test_s

# Train KNN on PCA data
```

```
knn_pca = KNeighborsClassifier(n_neighbors=7)
knn_pca.fit(X_train_pca, y_train)

print("Train score after PCA:", knn_pca.score(X_train_pca, y_train))
print("Test score after PCA:", knn_pca.score(X_test_pca, y_test))
```

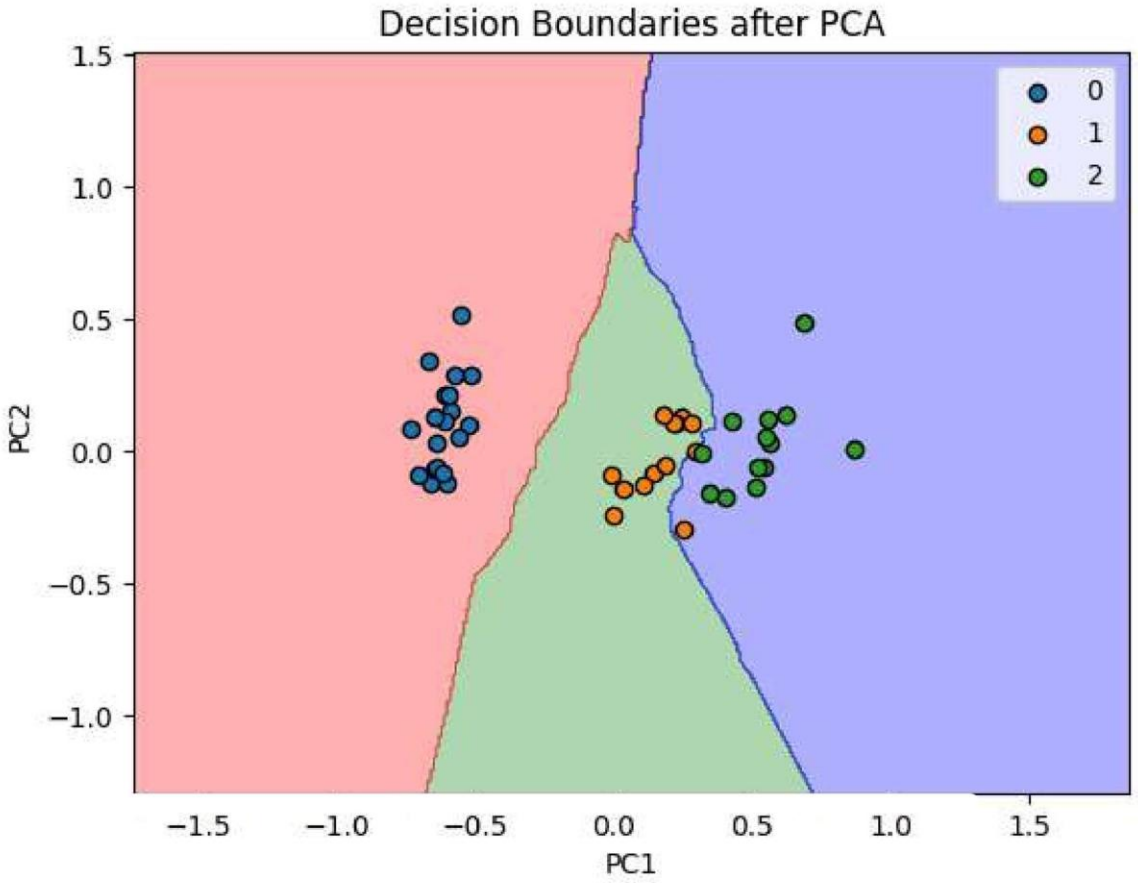
```
Explained Variance Ratio: [0.84136038 0.11751808 0.03473561 0.00638592]
```




```
Train score after PCA: 0.9523809523809523
Test score after PCA: 1.0
```

```
from matplotlib.colors import ListedColormap

# 2D plot if you use only 2 PCA components
X_2D = PCA(n_components=2).fit_transform(X_scaled)
X_train2D, X_test2D, y_train2D, y_test2D = train_test_split(X_2D, y, test_size=0
knn2D = KNeighborsClassifier(n_neighbors=7)
knn2D.fit(X_train2D, y_train2D)

# Meshgrid for plotting
X1, X2 = np.meshgrid(
    np.arange(X_test2D[:, 0].min() - 1, X_test2D[:, 0].max() + 1, 0.01),
    np.arange(X_test2D[:, 1].min() - 1, X_test2D[:, 1].max() + 1, 0.01)
)
plt.contourf(X1, X2, knn2D.predict(np.c_[X1.ravel(), X2.ravel()]).reshape(X1.sha
             alpha=0.3, cmap=ListedColormap(('red', 'green', 'blue')))

for i, j in enumerate(np.unique(y_test2D)):
    plt.scatter(X_test2D[y_test2D == j, 0], X_test2D[y_test2D == j, 1],
                label=j, edgecolor='k')

plt.title("Decision Boundaries after PCA")
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
plt.legend()
plt.show()
```



V. Conclusion

PCA effectively reduces dimensions with minimal loss of variance.

The model performance after PCA is comparable to the full-feature model.

Explained variance shows the first 2 components capture over 95% of the variance.

Dimensionality reduction helps in visualization and training time reduction.

It's important to normalize data before PCA to ensure all features are treated equally.